Mastering Terraform Replace and Taint Commands Guide

Cloud resources sometimes get corrupted or stuck in bad states without Terraform noticing configuration changes. Learning how to force-recreate specific resources helps you fix broken environments safely without touching healthy infrastructure.

What you'll learn:

1. Inspecting State with terraform state list
2. Target-Replacing with terraform apply -replace
3. Legacy Flagging with terraform taint
4. Zero Downtime with create_before_destroy

Aman Raj Singh

Cloud Operations Engineer @ iManage

linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Inspecting State with terraform state list

Before you can force-recreate a broken resource, you need its exact Terraform identifier. Running state list prints every tracked resource in your environment. Copy the exact resource name from the output to target it precisely without guessing.

```
$terraform state list
```

2

TIP 2 OF 4

Target-Replacing with terraform apply -replace

The modern standard for force-recreating a resource is the -replace flag. It tells Terraform to destroy and rebuild only the specified target during the apply execution. This avoids permanently editing state files and gives you instant single-command fixes.

```
$terraform apply -replace="aws_instance.web_app"
```

3

TIP 3 OF 4

Legacy Flagging with terraform taint

In older versions of Terraform, you marked resources as tainted to schedule them for recreation on the next run. While -replace is now preferred, knowing taint is essential for maintaining legacy pipelines. If you taint by mistake, use untaint to reverse it.

```
$terraform taint aws_instance.web_app
```

4

TIP 4 OF 4

Zero Downtime with create_before_destroy

By default, Terraform deletes an existing resource before building the replacement, causing temporary outage. Adding the create_before_destroy rule inside the resource lifecycle block forces Terraform to spin up the new infrastructure before terminating the old one.

```
resource "aws_instance" "web_app" {
  # config...
  lifecycle {
    create_before_destroy = true
  }
}
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Use -replace over legacy taint for safer single-step execution.
+ Always run terraform plan -replace="..." first to review planned destruction.
+ Target precise resource addresses to avoid unintended side effects.
+ Add create_before_destroy in lifecycle blocks to prevent user downtime.
+ Keep state backups enabled before forcing resource replacements in production.

Watch Out For

! Forgetting that terraform taint modifies your remote state file immediately without confirmation.
! Using -replace on database instances without checking if storage volumes are persistent.
! Omitting quotes around complex resource addresses causing shell parsing errors.
! Recreating resources that have dependent services without planning for IP or ID changes.

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh